

Name: Natalie Poche
Date: September 25, 2026
UFID: 1533-9442

Report - Bootloader Development

1 Introduction

The purpose of this lab assignment is to develop raw binary bootloader images for both x86 and ARM processor architectures to understand the basics of boot routines and the services they provide. A primary objective of this project is to highlight the stark differences between ARM and x64 boot routines and their respective levels of hardware standardization. In this activity, bootloaders were written for the Intel i386 platform (using the 486 virtual CPU) and the ARM “Akita” platform. The developed bootloaders are designed to read a null-terminated string from a memory-mapped logging device and transmit this message to a serial Universal Asynchronous Receiver-Transmitter (UART) output device before halting execution.

2 Design

The design phase involved writing assembly source code and utilizing a GCC toolchain—specifically utilizing cross-compilation options for the ARM target—to build raw binary images via Makefiles. To test and verify the bootloaders, the QEMU command-line emulator and GNU Debugger (gdb-multiarch) were used. QEMU’s loader driver was explicitly used to simulate the mapping of flash ROM into the memory space.

On the x86 platform, the system architecture utilizes a standardized Basic Input-Output System (BIOS) loaded from ROM to initialize basic input and output devices. Because of this, the first stage bootloader does not need to worry about hardware initialization or loading the boot block (the first 512 bytes) into RAM, as the BIOS automatically handles loading it into address `0x7c00`. The logging device is mapped directly to the beginning of conventional memory at `0x500`. Furthermore, the UART devices (COM ports) on x86 are accessed via the data bus using a base port address (e.g., `0x03F8` for COM1) rather than being memory-mapped.

Conversely, the ARM architecture lacks a standardized firmware boot sequence and does not feature standardized address mapping for devices. Therefore, the first-stage bootloader must be explicitly developed for the exact hardware configuration. On the Akita platform, the bootloader begins execution at the trap table at address `0x000`, the memory-mapped log device is located at `0xA3F00000`, and the UART device is memory-mapped at `0x40100000`. Before data can be transmitted, the ARM bootloader must manually initialize the CPU by modifying the Current Program Status Register (CPSR) to set supervisor mode (`0x13`), disable THUMB instructions, and mask regular and fast IRQs.

3 Results and Discussion

Following successful builds, an investigative experiment was conducted utilizing different CPU emulations in QEMU to test the portability of the bootloader images.

x86 Experiment

After successfully running the bootloader on the i386 platform (486 CPU), the same bootloader image was executed using the x86_64 emulator (qemu64 CPU).

1. **Result:** The machine completed the task successfully.

A Project Deliverables

As required by the project specifications, a screencast demonstrating the Makefiles, build process, and QEMU emulation of both bootloaders is provided below.

Screencast Link: https://youtu.be/C69_gjl16MY

B Code Snippets

Below are the source assembly files and build automation script imported directly from external code files using the `\lstinputlisting` command.

Listing 1: x86 Assembly Bootloader Source (x86.boot.s)

```
.code16 # Instructs the assembler to generate 16-bit instructions, because x86 systems
boot in 16-bit Real Mode for backwards compatibility
.global _start # Exposes the _start label globally so the linker can identify the
program's entry point

_start:
    /* Set string source index to log device memory map */
    mov $0x500, %si      # Log device convetional memory address 0x500 into the
                        # Source Index ($si) register, where the QMEU environment mapped the logging
                        # device

print_loop:
    /* Load byte from %ds:%si into %al and increment %si */
    lodsb # Loads a single byte from the memory address at %ds:%si into the %al
        # register and automatically increments %si to point to the next byte
    test %al, %al        # Performs a bitwise AND on the % al register with itself
        # to check if the loaded byte is the null terminator (0)
    jz halt_cpu          # Jumps to the halt_cpu label if the zero flag is set,
        # indicating the end of the log string was reached

wait_uart:
    /* Read LSR to check UART status */
    # Replace direct immediate access:
    # inb $0x3fd, %al Incorrect (truncates to 0xfd)

    # With register-based I/O:
    mov $0x3fd, %dx # Loads the I/O port address 0x3fd (COM1 base 0x3F8 + Line Status
        # Register offset 5) into the %dx register
    inb %dx, %al    # Reads a byte from the Lone Status Register port (in %dx) into %al
        # to check the current UART status
    test $0x20, %al # Chekc if THR is empty # Tests the 0x20 bit of the LSR to
        # see if the Transmit Holding Register (THR) is empty
    jz wait_uart    # Jumps back to wait_uart if the bit is not set, meaning
        # the THR is full and the program must keep waiting

send_uart:
    /* Retrieve the character and output to THR */
    mov -1(%si), %al # Retreives the current character from memory (offsetting the
        # previous %si increment) back into %al for output
    # Replace direct immediate access:
    # outb %al, $0x3f8 Incorrect (truncates to 0xf8)

    # With register-based I/O:
    mov $0x3f8, %dx # Loads the base COM1 port address (0x3F8), which maps to the
        # Transmit Holding Register on write, into %dx
    outb %al, %dx   # Outputs the character byte in %al to the COM1 THR port
        # mapped in %dx
```

```

    jmp print_loop          # Jumps back to print_loop to process and transmit the next
                           # character in the string

halt_cpu:
    hlt                    # Halts the CPU execution to stop the bootloader after the
                           # message is sent
    jmp halt_cpu           # Creates an infinite loop jumping back to hlt, ensuring
                           # the CPU stays halted if awakened by an interrupt

/* Pad remainder of boot sector and add boot signature */
.org 510 # Pads the boot sector with zeros up to byte 510 to fill out the standard
512-byte boot block format
.word 0xaa55 # Writes the 2-byte boot signature (little-endian 0xaa55) at the end to
signify to the BIOS that this block is bootable

```

Listing 2: ARM Assembly Bootloader Source (arm.boot.s)

```

.global _start // Exposes the _start label globally so the linker recognizes it as the
entry point

_start:
/* 1. CPU Initialization */
mrs r0, cpsr // Moves the Current Program Status Register (CPSR) into register r0
to read the CPU's current state
bic r0, r0, #0xff // Uses Bit Clear to clear out the lowest 8 bits (control byte)
of the CPSR stored in r0
orr r0, r0, #0xd3 // Uses Bitwise OR to set supervisor mode, disable THUMB
instructions, and mask IRQs by writing b11010011 (0xd3)
msr cpsr_c, r0 // Moves the modified control byte back into the CPSR_c to finalize
CPU initialization

/* 2. Setup Memory Addresses */
ldr r1, =0xA3F00000 /* Log device address */ // Loads the memory-mapped address
of the Akita log device into r1
ldr r2, =0x40100000 /* UART THR (Base) */ // Loads the base memory-mapped
address for the Akita UART Transmit Holding Register into r2
ldr r3, =0x40100014 /* UART LSR */ // Loads the memory-mapped address of the
UART Line Status Register (base 0x40100000 + 5 * 4 byte padding) into r3

print_loop:
/* Load byte and post-increment log pointer */
ldrb r4, [r1], #1 // Loads a single byte from the log memory address in r1 into
r4, then increments the pointer address in r1 by 1
cmp r4, #0 // Compares the loaded byte in r4 to 0 to check if it is
the null terminator marking the end of the string
beq halt_cpu // Branches to halt_cpu if the comparison is equal, exiting
the print loop

wait_uart:
/* Read LSR and check if THR is empty (0x20) */
ldr r5, [r3] // Loads the 32-bit Line Status Register value from memory into
r5
tst r5, #0x20 // Tests the 0x20 bit of the LSR to verify if the Transmit
Holding Register is empty and ready for data
beq wait_uart // Branches back to wait_uart if the bit is not set, creating
a polling loop until the THR is empty

send_uart:
/* Write character to THR */
strb r4, [r2] // Stores the character byte from r4 into the UART Transmit
Holding Register memory address held in r2
b print_loop // Branches back to print_loop to read and process the next byte

```

in the log

```
halt_cpu:
    b halt_cpu // Branches infinitely to itself, permanently halting further execution
```

Listing 3: Build Automation Script (Makefile)

```
all: i386_image.bin akita_image.bin # Sets the default target to build both the x86 and
    arm bootloader binary images

# x86 Build Process
i386_image.bin: x86_image.s
#     as -march=i386 x86_image.s -o x86.o
#     as --32 -march=i386 x86_image.s -o x86.o # Uses GNU assembler to generate a
#         32-bit object file targeting oldest common instruction set (i386)
#     ld -Ttext 0x7c00 --oformat binary x86.o -o i386_image.bin
#     ld -m elf_i386 -Ttext 0x7c00 --oformat binary x86.o -o i386_image.bin # Links
#         the x86 object file into a raw binary format with the text segment starting
#         at 0x7c00, standard address where BIOS loads the boot block

# ARM Build Process
akita_image.bin: akita_image.s
    arm-none-eabi-as -march=armv4t akita_image.s -o arm.o # Cross-compile the ARM
        source code into an object file targeting the ARMv4t architecture to
        maintain backward compatibility
    arm-none-eabi-ld -Ttext 0x0 arm.o -o arm.elf # Links the ARM object file into
        an ELF executable, setting the text section address to 0x0 because
        execution begins at the start of the trap table on this architecture
    arm-none-eabi-objcopy -O binary --only-section=.text arm.elf akita_image.bin #
        Extracts only the text section from the ELF file to create the final raw
        binary ARM bootloader image, avoiding inclusion of unnecessary ELF headers

KEEP_BIN := lemon_log.bin
ALL_BINS := $(wildcard *.bin)
BINS_TO_REMOVE := $(filter-out $(KEEP_BIN), $(ALL_BINS))

clean:
    rm -f *.o *.elf $(BINS_TO_REMOVE)
```